

Offline RL Algorithms

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Some principles for offline RL

- Many different methods, similar principles seem to be effective:

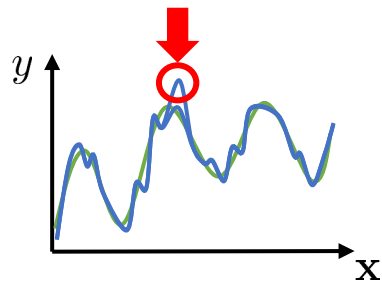
use value-based methods (i.e., Q-learning or Q-function actor-critic)

somehow fix the distributional shift problem

$$D_{\text{KL}}(\pi(\mathbf{a}|\mathbf{s})||\pi_{\beta}(\mathbf{a}|\mathbf{s})) \leq \epsilon$$

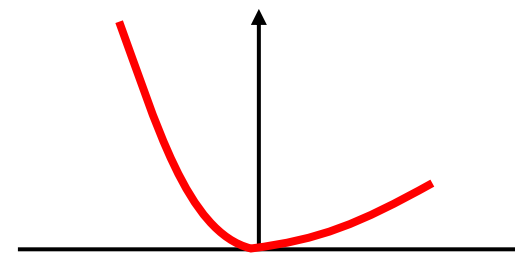
policy constraints

train the policy so that it
stays close to data



pessimism (e.g., CQL)

train the Q-function so that
it doesn't overestimate



avoid OOD actions in updates
(e.g., IQL)

set up the whole algorithm to
avoid out-of-sample actions

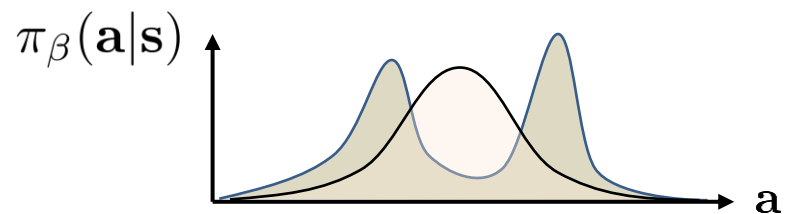
Part 1:

Policy constraints continued

Policy constraints review

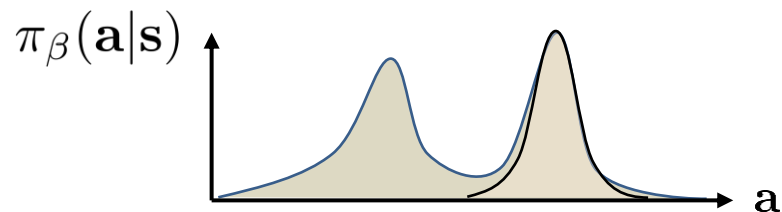
$$\theta \leftarrow \arg \max_{\theta} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\beta}) \leq \epsilon$$

$$D_{\text{KL}}(\pi_{\beta}(\mathbf{a}|\mathbf{s}) \parallel \pi_{\theta}(\mathbf{a}|\mathbf{s})) \quad \text{"forward KL"}$$

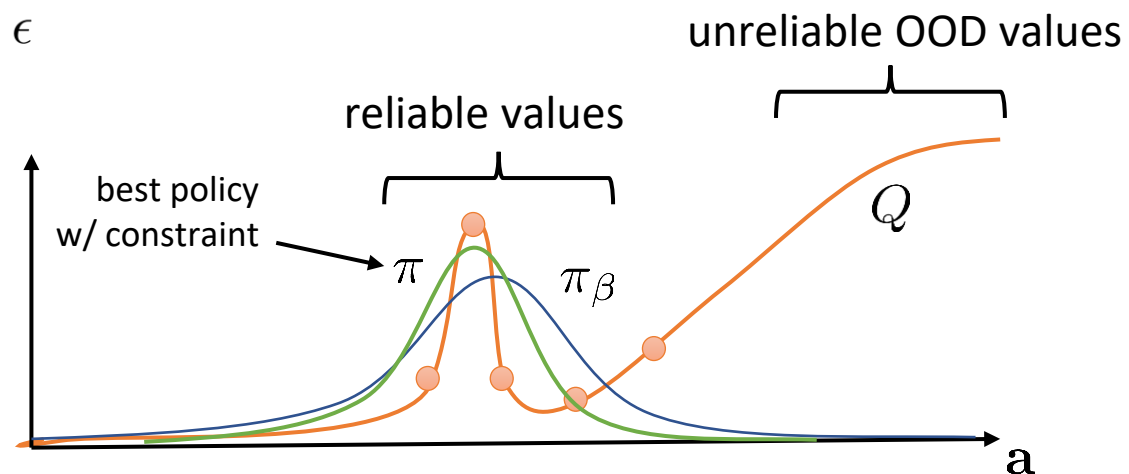


"mode covering"

$$D_{\text{KL}}(\pi_{\theta}(\mathbf{a}|\mathbf{s}) \parallel \pi_{\beta}(\mathbf{a}|\mathbf{s})) \quad \text{"reverse KL"}$$



"mode seeking"



keeping π_{θ} "close" to π_{β} limits distributional shift

Explicit policy constraint methods

How do we implement constraints?

1. Modify the actor objective

$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a})]$$

Lagrange multiplier

easy to compute and differentiate
for Gaussian or categorical policies

1a

$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a}) + \lambda \log \pi_{\beta}(\mathbf{a}|\mathbf{s}_i)] + \lambda \mathcal{H}(\pi_{\theta}(\mathbf{a}|\mathbf{s}_i))$$
$$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\beta}) = E_{\pi_{\theta}}[\log \pi_{\theta}(\mathbf{a}|\mathbf{s}) - \log \pi_{\beta}(\mathbf{a}|\mathbf{s})] = -E_{\pi_{\theta}}[\log \pi_{\beta}(\mathbf{a}|\mathbf{s})] - \mathcal{H}(\pi_{\theta}) \quad \text{“reverse KL”}$$

1b

$$\theta \leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a})] + \lambda \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i)$$
$$D_{\text{KL}}(\pi_{\beta} \parallel \pi_{\theta}) = -E_{\pi_{\beta}}[\log \pi_{\theta}(\mathbf{a}|\mathbf{s})] + \text{const} \quad \text{“forward KL”}$$

sometimes called *[method]+BC* (e.g., DDPG+BC, TD3+BC, SAC+BC, etc.)

generally not the *best* way to fix the problem, but can work well if done right

Explicit policy constraint methods

How do we implement constraints?

typically uses reverse KL + MaxEnt RL

2. Modify the reward function

$$D_{\text{KL}}(\pi_{\theta}(\mathbf{a}|\mathbf{s})||\pi_{\beta}(\mathbf{a}|\mathbf{s})) = -\mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})}[\log \pi_{\beta}(\mathbf{a}|\mathbf{s})] - \mathcal{H}(\pi_{\theta}(\mathbf{a}|\mathbf{s}))$$

$$\bar{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \lambda \log \pi_{\beta}(\mathbf{a}|\mathbf{s})$$

why might we *prefer* this?

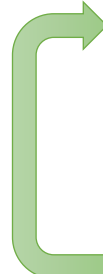
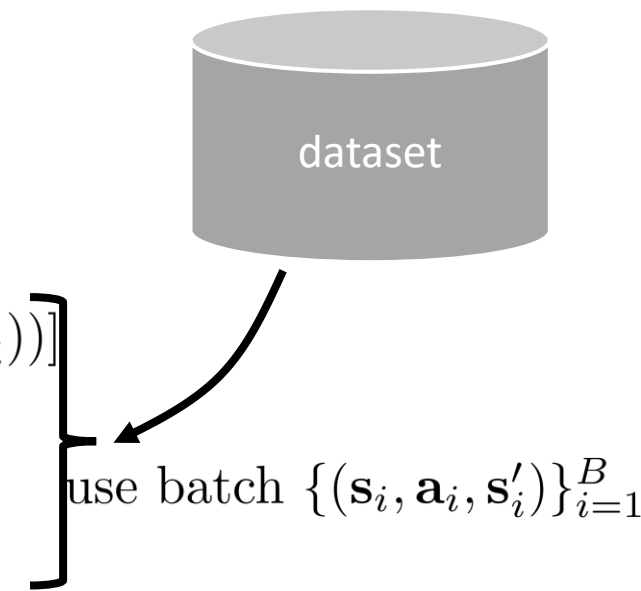
also accounts for **future** divergence

why might we *not prefer* this?

requires estimating the behavior policy

Example: BRAC-like algorithm

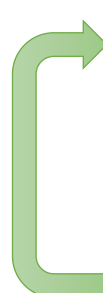
offline actor-critic algorithm (BRAC-like):

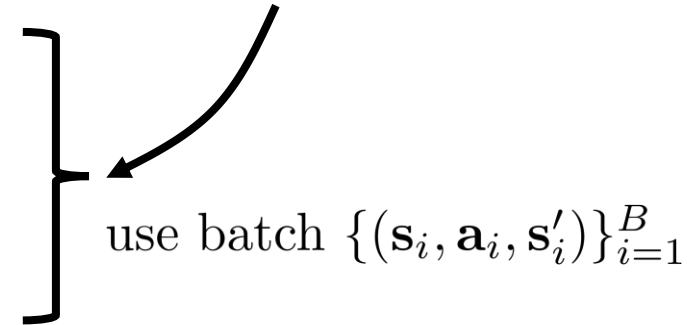
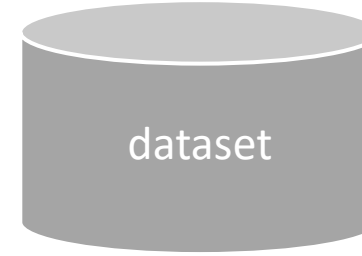
- 
1. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}') - \lambda D(\pi_\theta(\cdot | \mathbf{s}'_i), \pi_\beta(\cdot, \mathbf{s}'_i))]$
 2. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 3. $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})] - \lambda D(\pi_\theta(\cdot | \mathbf{s}_i), \pi_\beta(\cdot | \mathbf{s}_i))$
 4. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- 

for example $D(\pi_\theta, \pi_\beta) = D_{\text{KL}}(\pi_\theta \| \pi_\beta)$

Example: AC+BC-like algorithm

offline actor-critic algorithm (AC+BC-like):

- 
1. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
 2. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 3. $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})] + \lambda \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i)$
 4. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Implicit policy constraint methods

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg \max_{\pi} E_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi \parallel \pi_{\beta}) \leq \epsilon$$

$$\pi^*(\mathbf{a}|\mathbf{s}) = \frac{1}{Z(\mathbf{s})} \pi_{\beta}(\mathbf{a}|\mathbf{s}) \exp \left(\frac{1}{\lambda} A^{\pi}(\mathbf{s}, \mathbf{a}) \right)$$

straightforward to show via duality

See also:

Peters et al. (REPS)

Rawlik et al. (“psi-learning”)

...many follow-ups

approximate via **weighted** max likelihood!

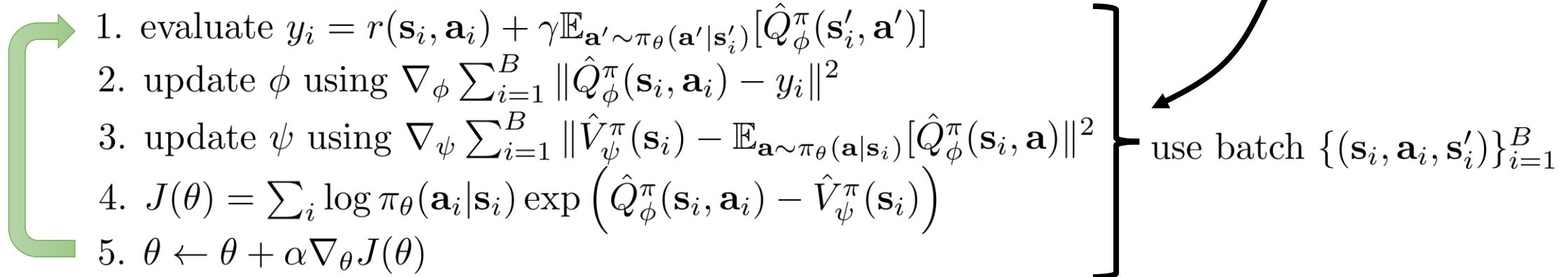
$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg \max_{\pi} E_{(\mathbf{s}, \mathbf{a}) \sim \pi_{\beta}} \left[\log \pi(\mathbf{a}|\mathbf{s}) \overbrace{\frac{1}{Z(\mathbf{s})} \exp \left(\frac{1}{\lambda} A^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a}) \right)}^{w(\mathbf{s}, \mathbf{a})} \right]$$

↑
samples from dataset
 $\mathbf{a} \sim \pi_{\beta}(\mathbf{a}|\mathbf{s})$

↖
critic can be used
to give us this

Example: AWAC-like algorithm

offline actor-critic algorithm (AWAC-like):

- 
1. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
 2. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 3. update ψ using $\nabla_\psi \sum_{i=1}^B \|\hat{V}_\psi^\pi(\mathbf{s}_i) - \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]\|^2$
 4. $J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \exp \left(\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi^\pi(\mathbf{s}_i) \right)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

can also just approximate $\hat{V}^\pi(\mathbf{s}_i) \approx \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})$ where $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)$

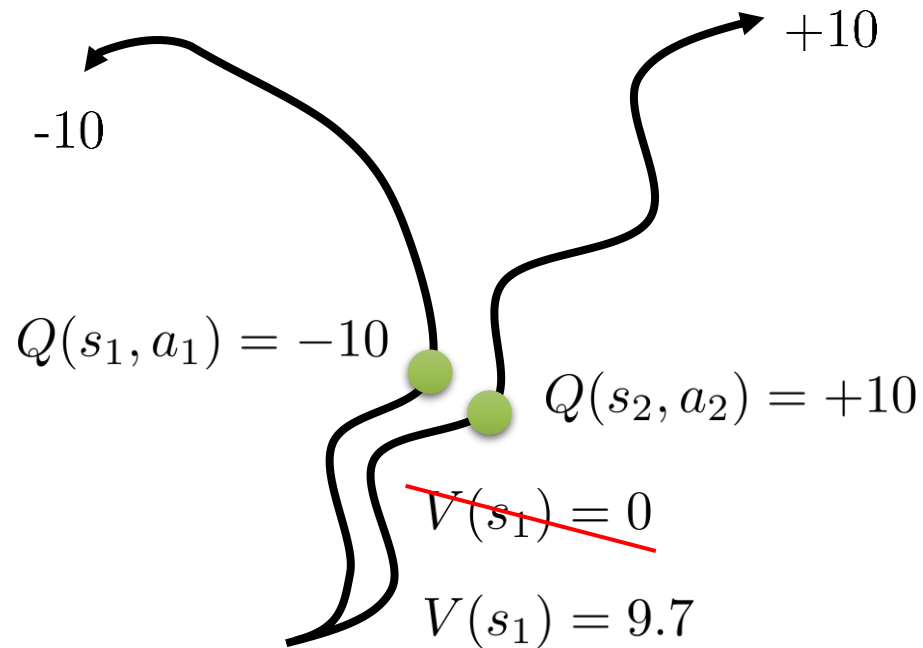
what are the *advantages* of this approach?

what are possible *problems* with this approach?

Part 2: Implicit Q-learning

Can we avoid *all* OOD actions in the Q update?

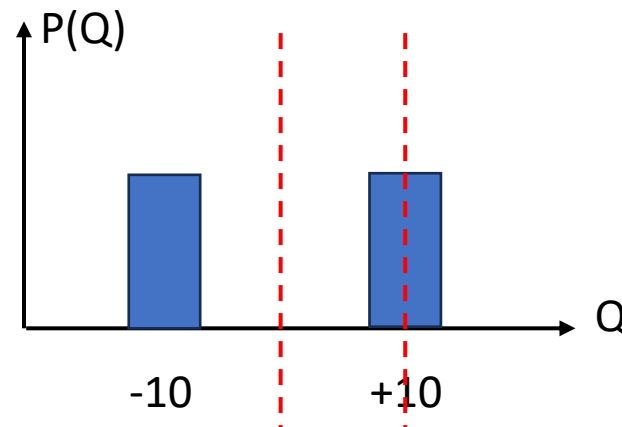
What if we didn't need to *ever* evaluate any action that is not in the dataset?



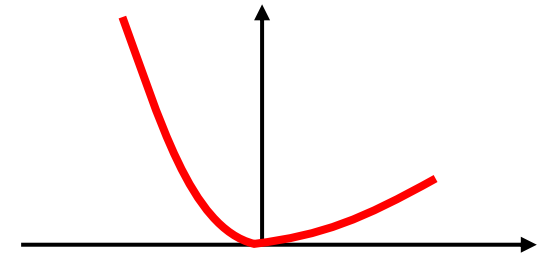
$$Q = \min_Q \sum_{(s,a,s') \in \mathcal{D}} \|Q(s,a) - [r(s,a) + \gamma V(s')]\|^2$$

$$V = \min_V \sum_{(s,a) \in \mathcal{D}} \|V(s) - Q(s,a)\|^2$$

$$V = \min_V \sum_{(s,a) \in \mathcal{D}} \ell_\tau(V(s) - Q(s,a))$$



mean upper quantile



Implicit Q-learning (IQL)

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \underbrace{E_{\mathbf{a}' \sim \pi_{\text{new}}} [Q(\mathbf{s}', \mathbf{a}')]]}_{V(\mathbf{s}')}$$

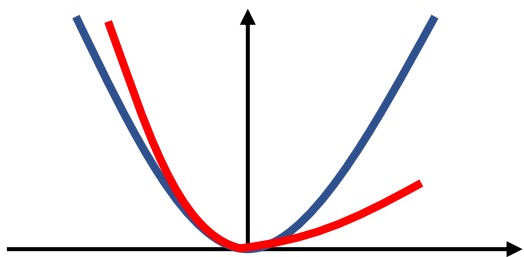
$V(\mathbf{s}')$ ← just another neural network

$$V \leftarrow \arg \min_V \frac{1}{N} \sum_{i=1}^N \ell(V(\mathbf{s}_i), Q(\mathbf{s}_i, \mathbf{a}_i))$$

e.g., MSE loss $(V(\mathbf{s}_i) - Q(\mathbf{s}_i, \mathbf{a}_i))^2$

this action comes from π_β
not from π_{new}

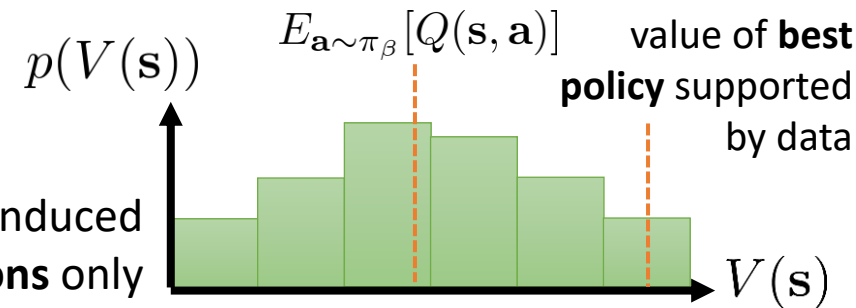
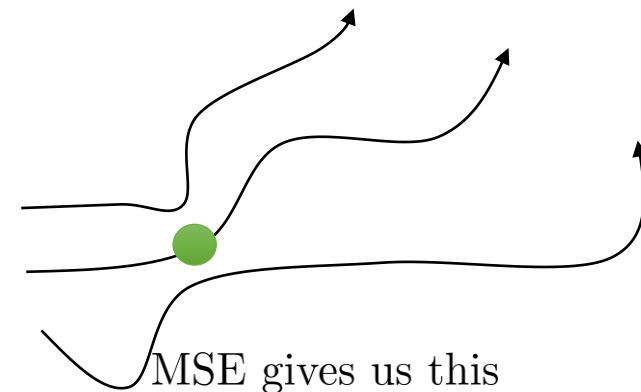
expectile: $\ell_2^\tau(x) = \begin{cases} (1 - \tau)x^2 & \text{if } x > 0 \\ \tau x^2 & \text{else} \end{cases}$



$$V(\mathbf{s}) \leftarrow \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s}, \mathbf{a})$$

$$\Omega(\mathbf{s}) = \{\mathbf{a} : \pi_\beta(\mathbf{a}|\mathbf{s}) \geq \epsilon\}$$

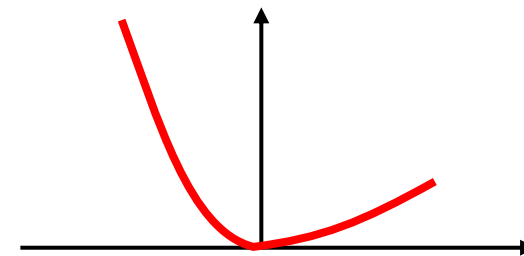
if we use ℓ_2^τ for big τ



Implicit Q-learning (IQL)

Q-learning with *implicit* policy improvement

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + V(\mathbf{s}') \quad V \leftarrow \arg \min_V \frac{1}{N} \sum_{i=1}^N \ell_2^\tau(V(\mathbf{s}_i), Q(\mathbf{s}_i, \mathbf{a}_i))$$



$$V(\mathbf{s}) \leftarrow \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s}, \mathbf{a})$$

$$\Omega(\mathbf{s}) = \{\mathbf{a} : \pi_\beta(\mathbf{a}|\mathbf{s}) \geq \epsilon\}$$



$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \max_{\mathbf{a}' \in \Omega(\mathbf{s}')} Q(\mathbf{s}', \mathbf{a}')$$

“implicit” policy

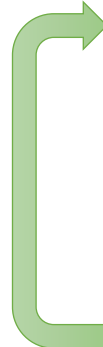
if we use ℓ_2^τ for big τ

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \delta(\mathbf{a} = \arg \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s}, \mathbf{a}))$$

Now we can do value function updates without ever risking out-of-distribution actions!

Implicit Q-learning (IQL)

offline actor-critic with IQL:

- 
1. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\psi(\mathbf{s}'_i)$
 2. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 3. update ψ using $\nabla_\psi \sum_{i=1}^B \ell_2^\tau(\hat{V}_\psi(\mathbf{s}_i) - \hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i))$
 4. $J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \exp(\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi(\mathbf{s}_i))$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- 
- 
- use batch $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$

there are no expectations under π_θ anywhere

critic is also completely independent of actor

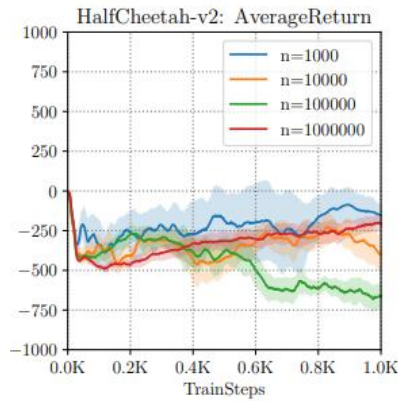
hence implicit “Q-learning,” not really “actor-critic”

Intermission

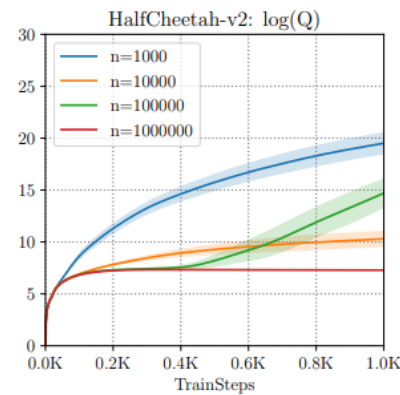
Part 3:

Conservative Q-learning

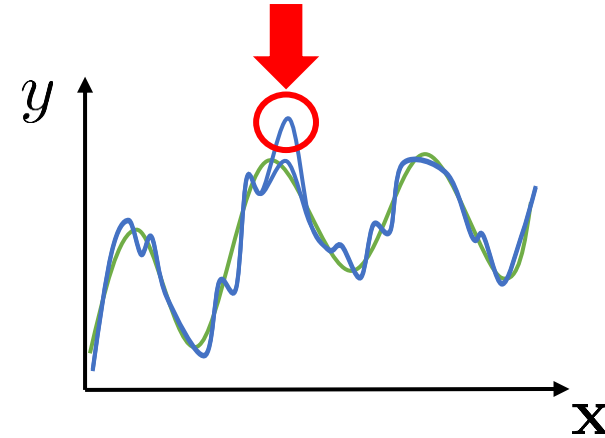
Conservative Q-learning (CQL)



how well it does



how well it *thinks*
it does (Q-values)



$$\hat{Q}^\pi = \arg \min_Q \max_\mu \alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \quad \left. \vphantom{\arg \min_Q} \right\} \text{ term to push down big Q-values}$$

$$\text{regular objective} \quad \left\{ + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2 \right] \right\}$$

can show that $\hat{Q}^\pi \leq Q^\pi$ for large enough α

true Q-function

Conservative Q-learning (CQL)

A *better* bound:

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \left\{ \begin{array}{l} \text{always pushes Q-values down} \quad \text{push up on } (\mathbf{s}, \mathbf{a}) \text{ samples in data} \\ \alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha E_{(\mathbf{s}, \mathbf{a}) \sim D} [Q(\mathbf{s}, \mathbf{a})] \\ + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2 \right] \end{array} \right\} \mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$$

no longer guaranteed that $\hat{Q}^\pi(\mathbf{s}, \mathbf{a}) \leq Q^\pi(\mathbf{s}, \mathbf{a})$ for all $(\mathbf{s}, \mathbf{a})$

but guaranteed that $E_{\pi(\mathbf{a}|\mathbf{s})}[\hat{Q}^\pi(\mathbf{s}, \mathbf{a})] \leq E_{\pi(\mathbf{a}|\mathbf{s})}[Q^\pi(\mathbf{s}, \mathbf{a})]$ for all $\mathbf{s} \in D$

Conservative Q-learning (CQL)



1. Update $\hat{Q}^\pi$ w.r.t. $\mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$ using $\mathcal{D}$
2. Update policy π

if actions are discrete:

$$\pi(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max_{\mathbf{a}} \hat{Q}(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

if actions are continuous:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \sum_i E_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}(\mathbf{s}_i, \mathbf{a})]$$

Conservative Q-learning (CQL)

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \underbrace{\alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]}_{\text{regularization}} - \alpha E_{(\mathbf{s}, \mathbf{a}) \sim D} [Q(\mathbf{s}, \mathbf{a})] - \mathcal{R}(\mu) + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2 \right] \Bigg\} \mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$$

common choice: $\mathcal{R} = E_{\mathbf{s} \sim D} [\mathcal{H}(\mu(\cdot|\mathbf{s}))]$ maximum entropy regularization

optimal choice: $\mu(\mathbf{a}|\mathbf{s}) \propto \exp(Q(\mathbf{s}, \mathbf{a}))$

$$E_{\mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] = \log \underbrace{\sum_{\mathbf{a}} \exp(Q(\mathbf{s}, \mathbf{a}))}$$

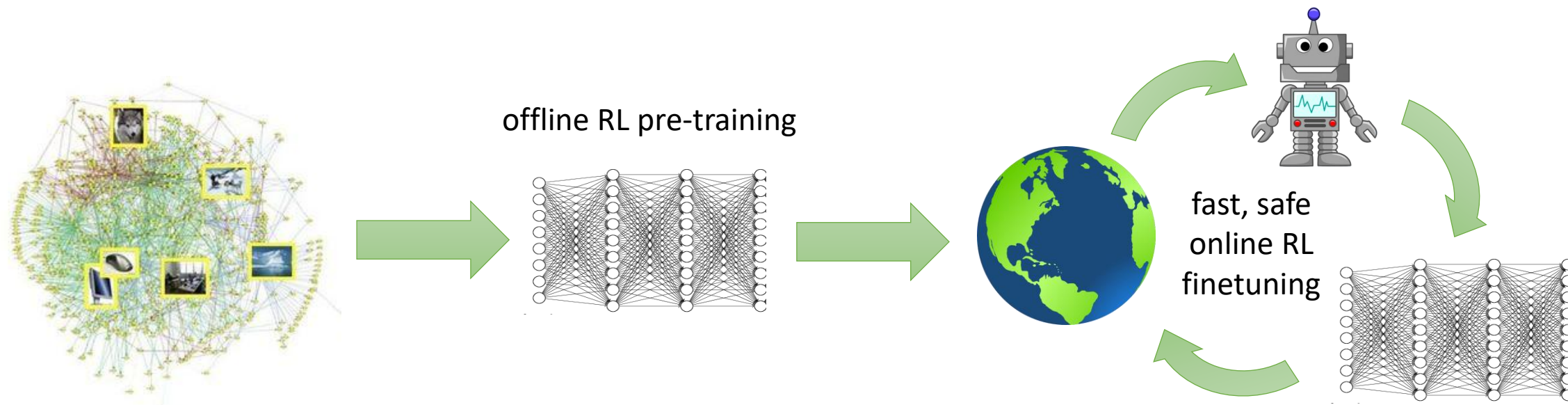
for discrete actions: just calculate directly

for continuous actions: use importance sampling to estimate $E_{\mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]$

Part 4:

Offline-to-online RL

The big picture

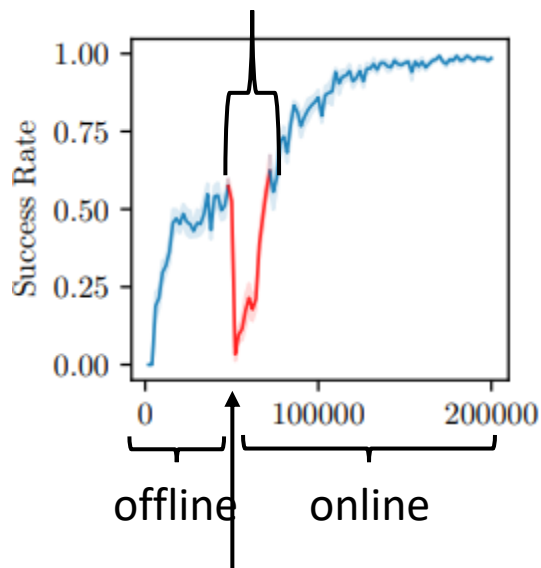


What's the problem?

Example: offline to online RL
with CQL

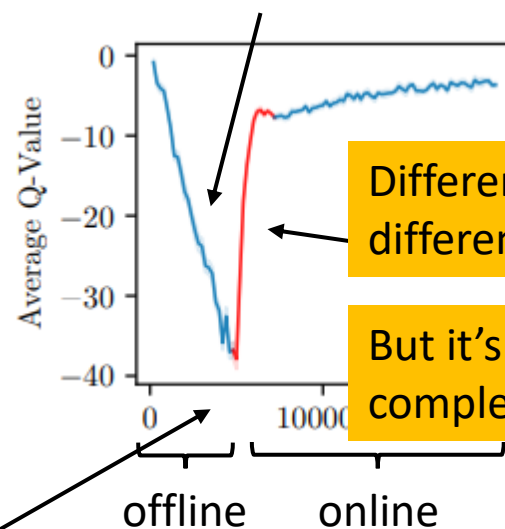
$$\hat{Q}^{\pi} = \arg \min_Q \max_{\pi} \alpha E_{\mathbf{s} \sim D, \mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha E_{\mathbf{s}, \mathbf{a} \sim D} [Q(\mathbf{s}, \mathbf{a})] \\ + E_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} \left[(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + E_{\pi} [Q(\mathbf{s}', \mathbf{a}')]))^2 \right]$$

this period wasted recovering
offline performance



online training starts here (at 50k steps)

underestimating too much here



Different offline RL algorithms have
different versions of this problem

But it's very hard to fix this
completely!

The quest for the one ultimate algorithm

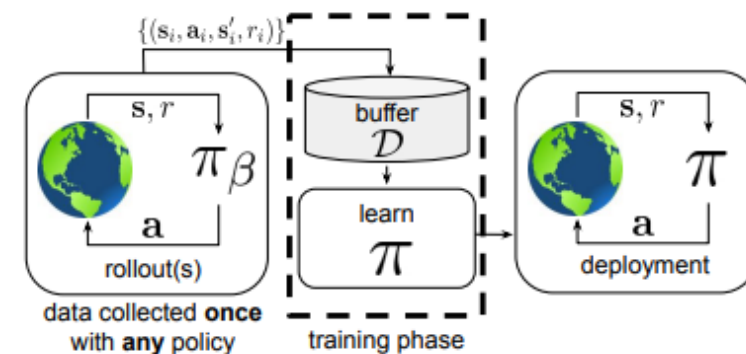
The problem:

offline phase:

stay close to π_β

conservative values (e.g., $\hat{Q}^\pi \leq Q^\pi$)

lots of gradient steps on the same data



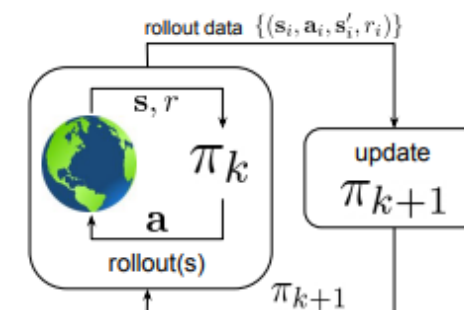
very hard to figure out one algorithm
that does both of these things well

online phase:

improve as much as possible over π_β

optimistic values (e.g., $\hat{Q}^\pi \geq Q^\pi$) this is important for exploration, more on that later

learn as quickly as possible

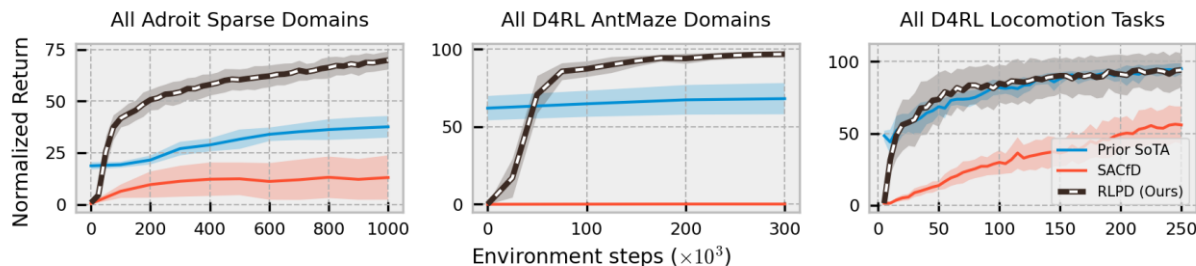
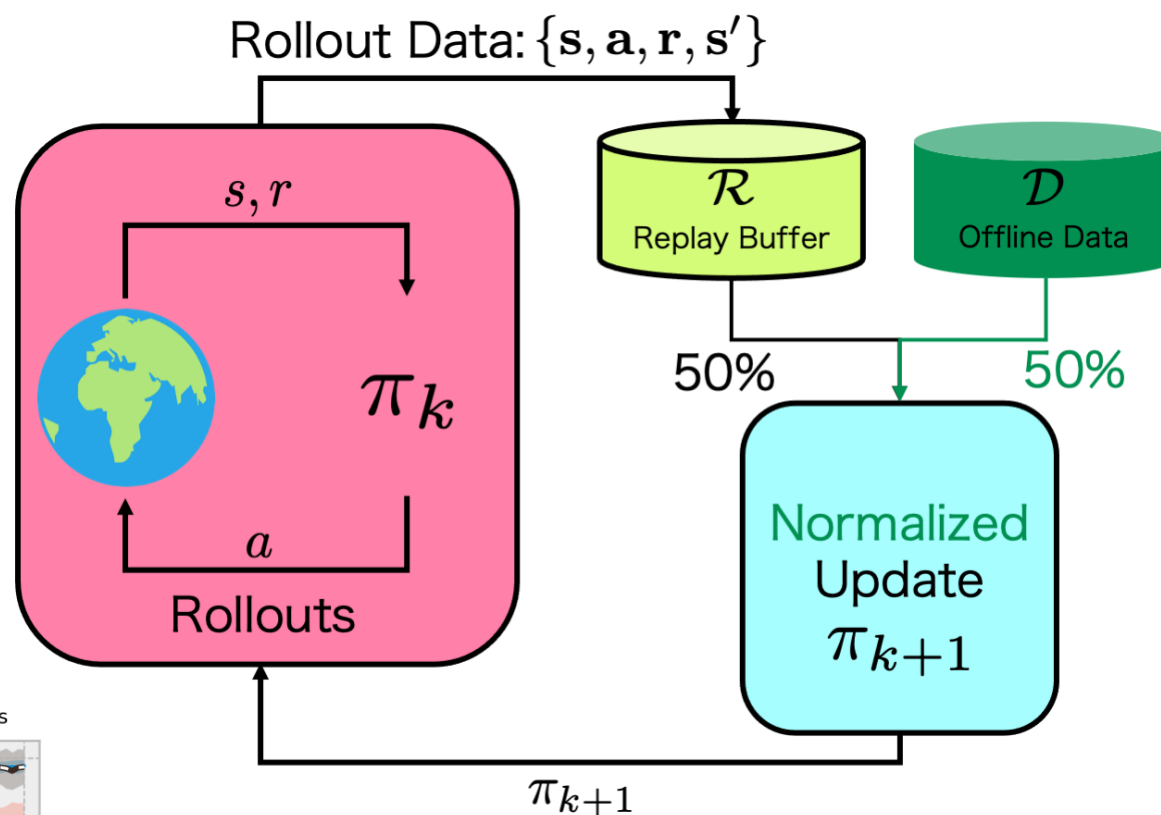


An embarrassingly effective method

1. Initialize two buffers: an online replay buffer and an offline data buffer
2. Initialize the value function (and actor) **from scratch** (no pretraining!)
3. Run online RL, and for every batch, sample half of it from the offline data, and half of it from the online data replay buffer

This is deeply unsatisfying

This is really hard to beat with any “proper” offline to online method

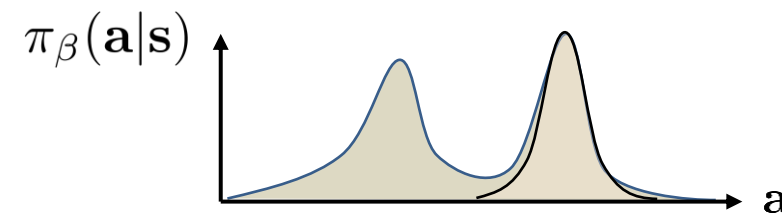


An unlikely hero?

For some reason, algorithms that use diffusion models or flow matching to represent the actor tend to make this work better

I don't know *exactly* why that's the case

Some possibilities...



in the online case, optimal $\pi_{\theta}(\mathbf{a}|\mathbf{s})$ is deterministic

(no need to capture multiple modes)

in the offline case, just capturing the best mode in the data is fine

if we can represent multiple modes, it's easier to learn

surprisingly, many of the offline-to-online RL methods we'll discuss only train the diffusion/flow model with supervised learning!

maybe for offline-to-online, it helps to track all the modes in online mode

then “double down” on the best one in online mode

Aside

Why is it hard to use diffusion as the actor in RL?

$$J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})]$$



requires either $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s})$

or backprop through the diffusion process (BPTT)

This *is* possible, see for example:

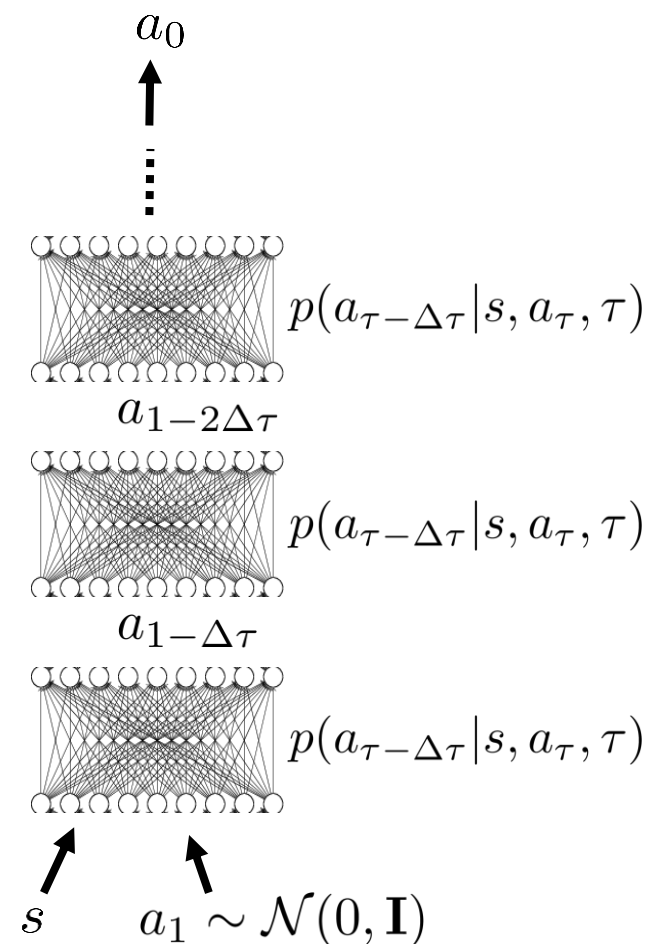
Black et al. "Training diffusion models with reinforcement learning."

Ren et al. "Diffusion policy policy optimization."

but it's a bit difficult

that said, maybe avoiding this is exactly what
allows diffusion models to work so well for
offline-to-online RL

diffusion or flow



IDQL: simple offline-to-online RL with diffusion

1. train $\hat{Q}_\phi(\mathbf{s}, \mathbf{a})$ using IQL (without any actor) $\left\{ \begin{array}{l} 1. \text{ update } \phi \text{ using } \nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2 \\ 2. \text{ update } \psi \text{ using } \nabla_\psi \sum_{i=1}^B \ell_2^\tau(\hat{V}_\psi(\mathbf{s}_i) - \hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i)) \\ 3. J(\theta) = \sum_i \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \exp(\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi(\mathbf{s}_i)) \end{array} \right.$
2. train $\pi_\theta(\mathbf{a}|\mathbf{s})$ as a diffusion/flow model with BC $\pi_\theta(\mathbf{a}|\mathbf{s}) \approx \pi_\beta(\mathbf{a}|\mathbf{s})$
3. at test time, sample $\{\mathbf{a}_1, \dots, \mathbf{a}_K\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ at each step
pick $\arg \max_{\mathbf{a}_k} \hat{Q}_\phi(\mathbf{s}, \mathbf{a}_k)$ no OOD actions because $\pi_\theta(\mathbf{a}|\mathbf{s}) \approx \pi_\beta(\mathbf{a}|\mathbf{s})$

why is this good?

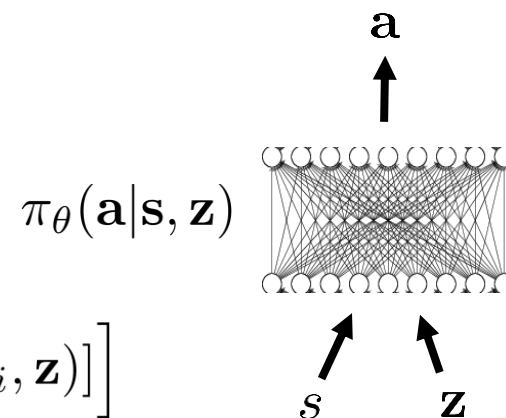
why is this bad?

FQL: actor that stays “close” to diffusion model

1. train $\pi_{\text{flow}}(\mathbf{a}|\mathbf{s}, \mathbf{z})$ as a diffusion/flow model with BC
2. run offline actor-critic with a special BC regularizer

$$\pi_{\text{flow}}(\mathbf{a}|\mathbf{s}) \approx \pi_{\beta}(\mathbf{a}|\mathbf{s})$$

this actor is **not** a diffusion model! It's just a regular neural net that takes in $\mathbf{z}$

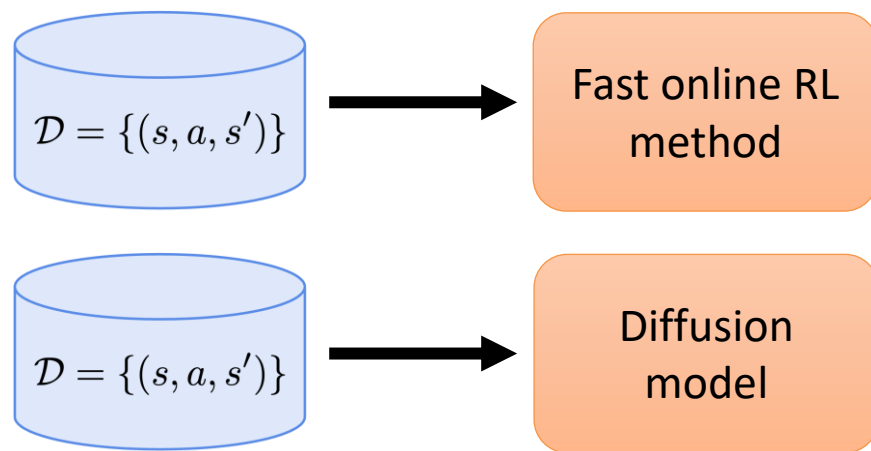


$$J(\theta) = \sum_i \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[\mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i, \mathbf{z})} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})] + \mathbb{E}_{\mathbf{a} \sim \pi_{\text{flow}}(\mathbf{a}|\mathbf{s}, \mathbf{z})} [\lambda \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i, \mathbf{z})] \right]$$

clone the action π_{flow} would take *for that value of $\mathbf{z}$*

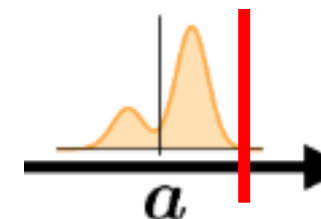
why is this good?

Diffusion steering



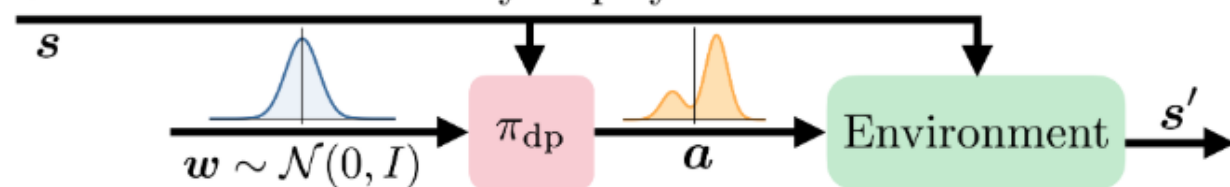
The problem: out of distribution actions

$$\max_{\pi} E_{a \sim \pi} [Q(s, a)]$$



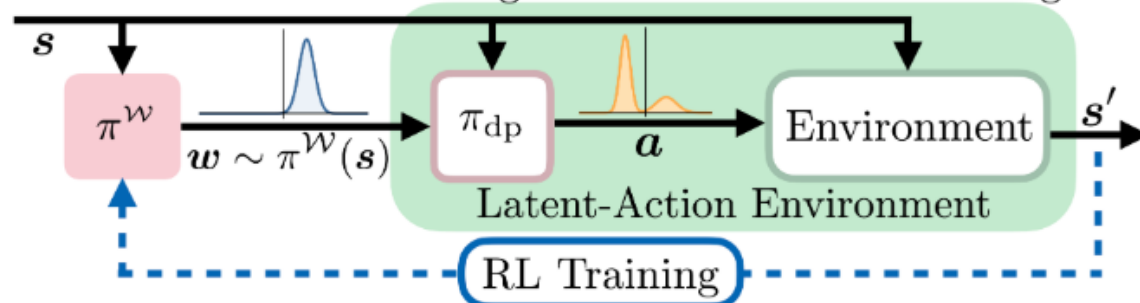
Key idea: run very efficient *online* RL (e.g., soft actor-critic) in the *latent space* of a diffusion model (e.g., DDIM or flow matching)

Standard Diffusion Policy Deployment



every noise sample is *in distribution*
(which means we can safely use offline data!)

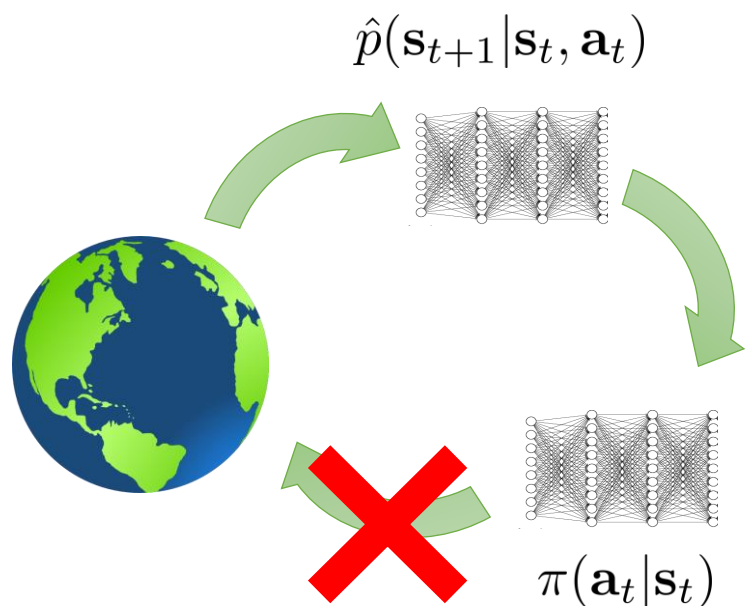
DSRL: Diffusion Steering via Reinforcement Learning



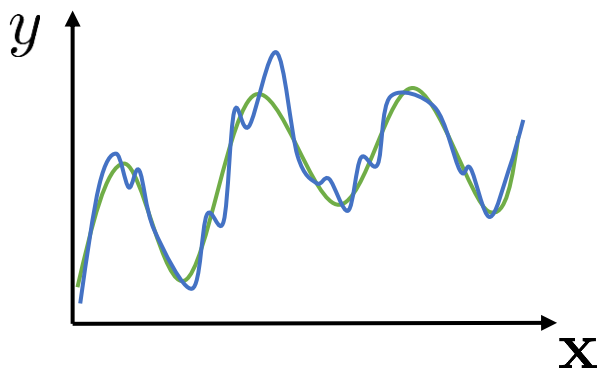
Part 5:

Model-based offline RL

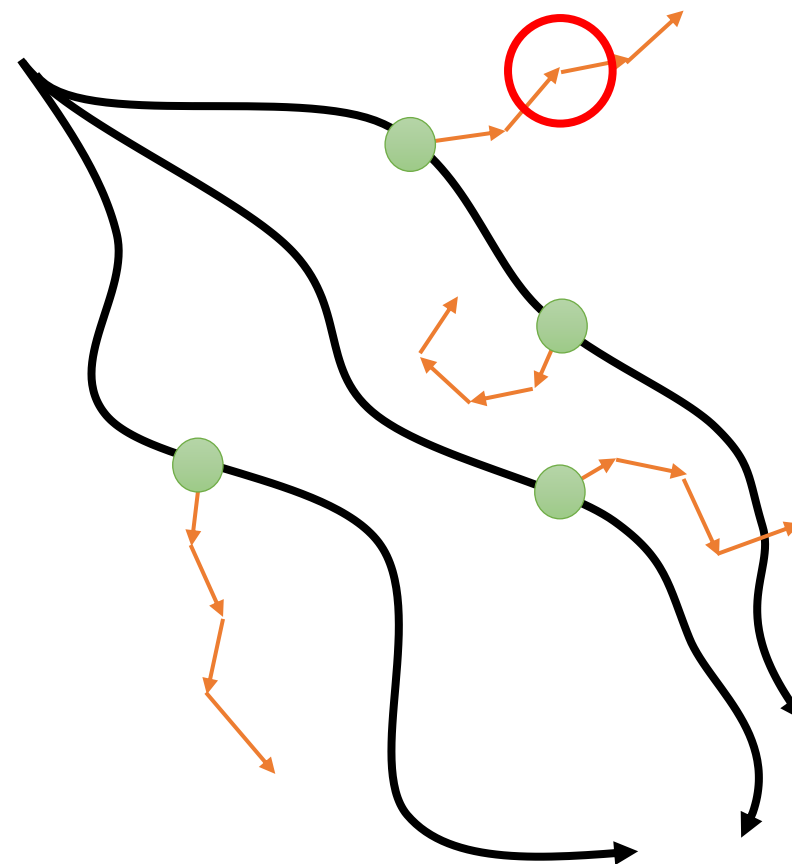
How does model-based RL work?



what goes wrong when we can't collect more data?



...so the model's predictions are invalid
these states are OOD



the model answers "what if" questions

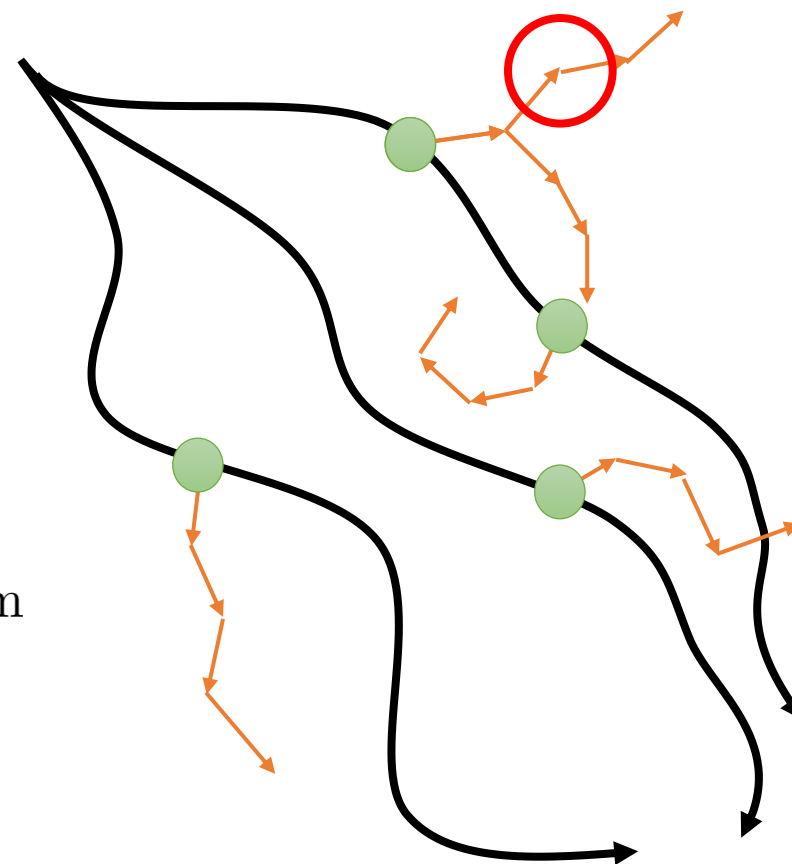
MOPO: Model-Based Offline Policy Optimization

solution: “punish” the policy for exploiting

$$\tilde{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) - \lambda u(\mathbf{s}, \mathbf{a})$$

← uncertainty penalty

...and then use any existing model-based RL algorithm



COMBO: Conservative Model-Based RL

Basic idea: just like CQL minimizes Q-value of policy actions, we can minimize Q-value of model state-action tuples

state-action tuples from the model

$$\begin{aligned} \hat{Q}^{k+1} \leftarrow \arg \min_Q & \beta \left(\mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \rho(\mathbf{s}, \mathbf{a})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} [Q(\mathbf{s}, \mathbf{a})] \right) \\ & + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim d_f} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{B}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right]. \end{aligned} \quad (4)$$

Intuition: if the model produces something that looks clearly different from real data, it's easy for the Q-function to make it look bad

